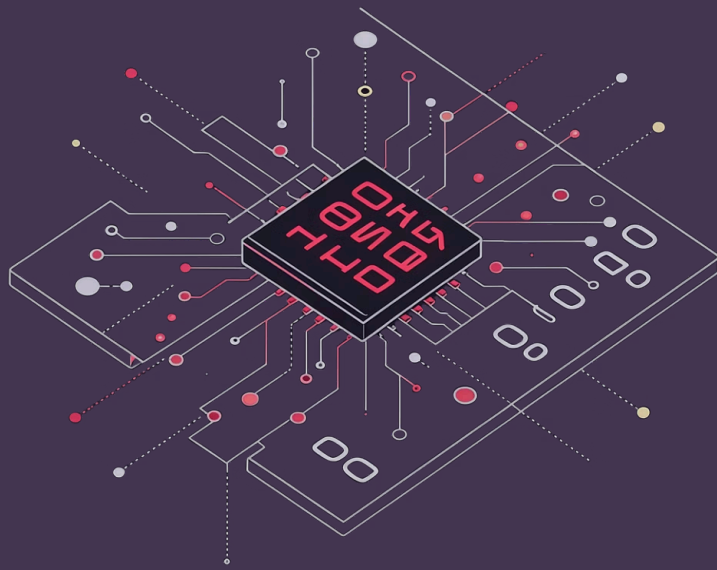
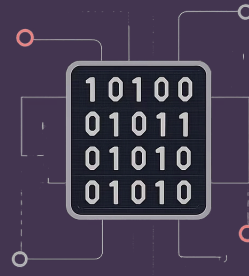




8-Bit Signed Binary: Sign-Magnitude Representation

A digital logic project implementing signed integer representation using Logisim — covering circuit design, methodology, and validation.



8-bit Signed Binary Project: A Technical Evaluation

This presentation outlines the development and analysis of an 8-bit signed binary system, designed for precise numerical operations and robust implementation.

- Problem Statement & Project Objectives
- System Design & Implementation Details
- Circuit Demonstration & Performance Analysis
- Code Repository & Technical Documentation

Problem Statement

The Core Limitation

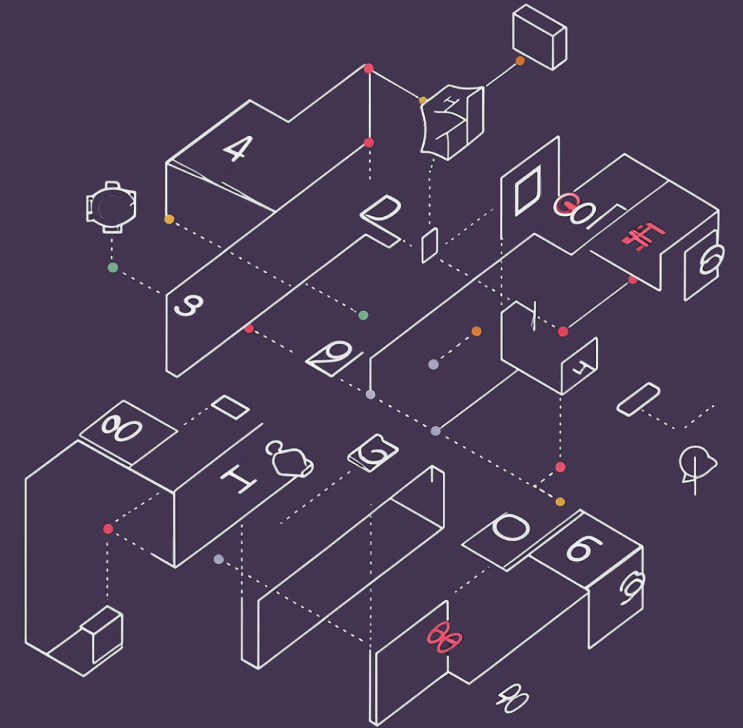
Standard unsigned binary cannot represent negative numbers — every bit pattern maps to a non-negative value only.

Sign-Magnitude Solution

The **MSB (Most Significant Bit)** acts as a sign flag: **0 = positive**, **1 = negative**. The remaining 7 bits encode the magnitude.

Representable Range

For 8-bit Sign-Magnitude, the valid range spans **-127 to +127**, with two representations for zero (00000000 and 10000000).



Design & Methodology

Magnitude Calculation

Process lower 7 bits to compute absolute value.

Output Formatting

Combine sign and decimal display format.

Sign Bit Extraction

Isolate the MSB to determine polarity.

Test Case Validation

Verify positive, negative, and edge cases.

Sign Bit Extraction

Isolate the MSB to determine the number's polarity before any arithmetic processing.

Magnitude Calculation

Lower 7 bits are passed directly to compute the absolute value independently of the sign.

Test & Validation

Edge cases include 0, -0, ± 127 to ensure correctness across the full range.

Logic Implementation Details

Our Logisim circuit translates 8-bit sign-magnitude binary into decimal values through distinct logical blocks.

1

Input Decoding

An 8-bit input bus is split: **1 bit for sign, 7 bits for magnitude**. This allows parallel processing of sign and value.

2

Sign Logic

The MSB drives a control signal. If 0, the output is positive; if 1, the output is negative.

3

Magnitude Conversion

The 7-bit magnitude is converted to its decimal equivalent using a standard **binary-to-decimal decoder** module in Logisim.

4

Output Display

A custom **7-segment display driver** (or similar component) renders the decimal magnitude, with the sign applied to the final output.

Logisim Circuit Demo

1

Input Stage

8-bit input bus labeled B7 to B0, with B7 as the sign bit and B6–B0 as magnitude.

2

Sign Detection

The MSB drives polarity output: 0 for positive, 1 for negative.

3

Magnitude Path

The lower 7 bits pass through decoder and display logic to show the absolute value.

4

Live Walkthrough

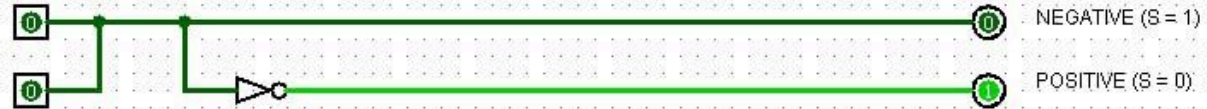
Example:

10000101 →

negative 5. Step through inputs to verify outputs in real time.

A. SIGN BIT DETECTION

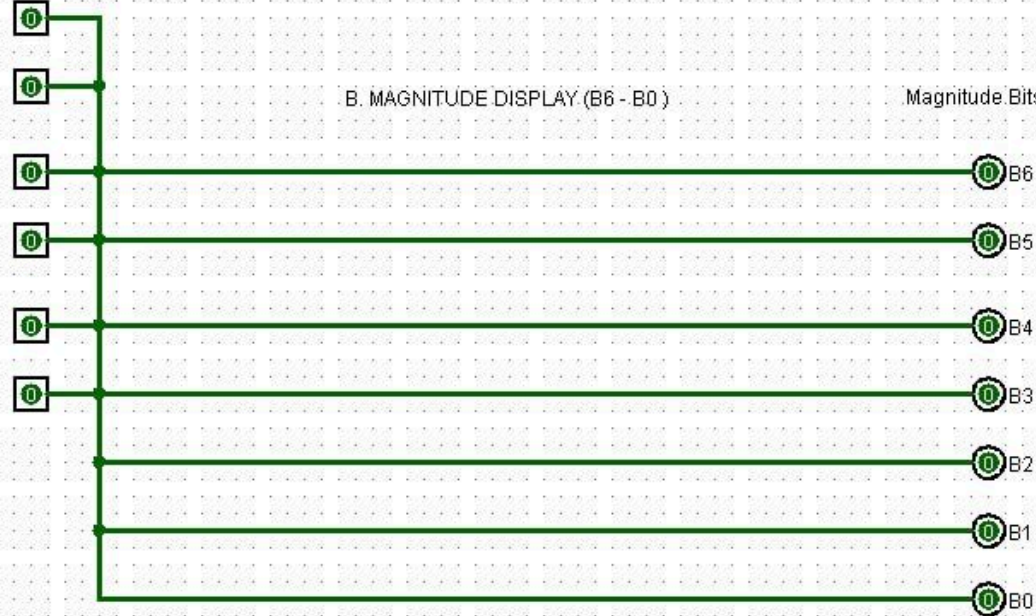
Sign Output

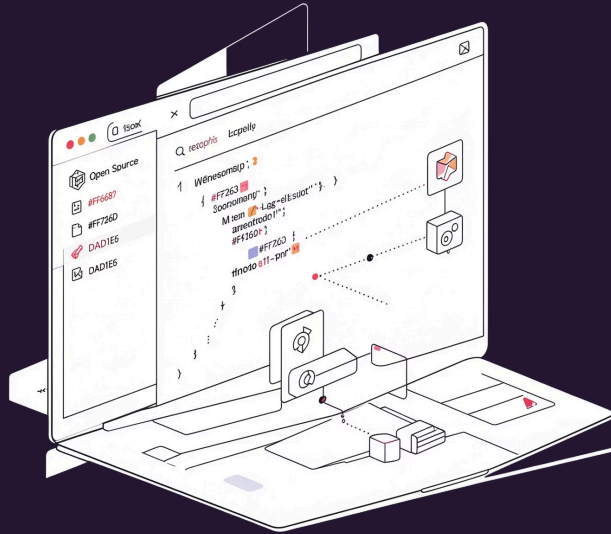


B. MAGNITUDE DISPLAY (B6 - B0)

Magnitude Bits

Binary Weight





GitHub Repository

All project assets are version-controlled and documented for reproducibility and peer review.

📁 **Repository:** <https://github.com/bobbamahima12/KLH-FED-2026-Session13-Batch-no---3.git>

1 Circuit Files

.circ files with documented logic gates and labeled components throughout.

2 README Documentation

Project overview, design rationale, Sign-Magnitude explanation, and usage instructions.

3 Test Cases & Expected Outputs

Comprehensive validation table covering positive, negative, zero, and edge-case inputs.

Advantages & Limitations of Sign-Magnitude

Advantages

- Intuitive & Easy to Understand: Sign bit is straightforward (0 = positive, 1 = negative)
- Symmetric Range: Equal representation of positive and negative numbers (-127 to +127)
- Simple Sign Detection: Determining sign requires only checking the MSB
- Direct Magnitude Access: Absolute value is directly readable from lower 7 bits

Limitations

- Dual Zero Problem: Both 00000000 and 10000000 represent zero, wasting one bit pattern
- Arithmetic Complexity: Addition/subtraction require different logic paths depending on signs
- Inefficient Hardware: Requires separate sign-checking logic; more gates than Two's Complement
- Limited Industry Use: Most modern systems use Two's Complement instead

Conclusion

This project demonstrates a complete and functional 8-bit Sign-Magnitude implementation in Logisim, combining binary representation theory with practical digital logic design.

Key Takeaways

- Successfully implemented 8-bit Sign-Magnitude representation in Logisim
- Demonstrated understanding of binary representation and digital logic design
- Validated circuit functionality across positive, negative, and edge cases
- Documented design decisions and trade-offs for academic evaluation

Overall, the project shows strong educational value and technical rigor by turning core computer architecture concepts into a working circuit model.

Thank You

Team Members: M Hiteshram (2620030066)

kevin (2620030078)

M. Haasini (2620030081)

Mahima (2620030076)